

Study of Fractional Differential Models

Caputo IVP, Discretization, and PECE Implementation

Vedant Jawandhia
Guide: Prof. Rajesh Kumar

Department of Mathematics, BITS Pilani

September 29, 2025

Slide 1: A brief history of fractional calculus & models

- **Origins (1695):** Leibniz asks what a “ $\frac{1}{2}$ -order derivative” would mean; L'Hôpital correspondence hints at general orders.
- **18th–19th c.:** Contributions by Euler, Fourier, Laplace; **Riemann** and **Liouville** formalize fractional *integrals* and *derivatives*.
- **20th c.:** **Caputo** derivative popularized for initial-value problems with classical (integer) initial conditions (ICs).
- **Modern era:** FDEs model **memory/hereditary** effects (viscoelasticity, anomalous diffusion, biology, finance). Growth of **numerical methods** and **software** (e.g., Garrappa's fde12).

Key idea: Fractional operators encode *history* via power-law kernels, unlike classical ODEs.

Slide 2: Types of FDEs & how they differ (plus methods)

Common fractional operators

- **Caputo derivative** ${}^C D_t^\alpha y(t)$: preferable for IVPs with standard ICs $y^{(k)}(0)$.
- **Riemann–Liouville (RL) derivative** and **RL integral** I_t^α : natural in integral formulations; ICs are less physical.
- **Grünwald–Letnikov** form: limit of fractional finite differences; basis for GL/L1 schemes.

Problem classes

- **Single-term** Caputo FDE: ${}^C D_t^\alpha y = f(t, y)$.
- **Multi-term** Caputo FDE: $\sum_{q=1}^Q \lambda_q {}^C D_t^{\alpha_q} y = f(t, y)$.
- **Linear vs Nonlinear**: linear if f is affine in y , else nonlinear (requires iterative solvers).

Representative numerical methods

- **Linear (multistep) method**: *Lubich's Convolution Quadrature* (FLMM/BDF2) for linear (and mildly nonlinear) FDEs; stable and high-order for linear systems.
- **Nonlinear method**: *Adams–Bashforth–Moulton PECE* (predictor–corrector) for Caputo IVPs (your chosen method; see next slides).

Slide 3: Why FDEs? Relevance & limitations of ODEs

Why FDEs are needed

- Many real systems exhibit **memory** or **hereditary** behavior ($\Rightarrow$ response depends on the entire past).
- Fractional models produce **power-law tails** and **anomalous diffusion** (sub/super-diffusion), matching experiments better than ODEs/PDEs.
- Solutions often involve the **Mittag–Leffler** function E_α , a natural generalization of the exponential.

Limitations of classical ODEs

- Assume **Markovian** (memoryless) dynamics; typical decay is exponential e^{-t} .
- Struggle to capture **long-range temporal correlations** and **nonlocal** effects observed in real data.

Applications (2–3 examples)

- **Viscoelasticity**: stress–strain with memory (polymers, tissues).
- **Anomalous transport**: subdiffusion in porous media; charge transport in complex materials.
- **Biology/Pharmacokinetics**: long-memory ion channels; fractional kinetics in drug absorption/elimination.

The Challenge: Solving Fractional DEs

We start with a fractional initial value problem for $0 < \alpha < 1$:

$${}^c D_t^\alpha y(t) = f(t, y(t)), \quad \text{with } y(0) = y_0.$$

Recall: Caputo Derivative Definition

The operator ${}^c D_t^\alpha$ is defined by an integral:

$${}^c D_t^\alpha y(t) = \frac{1}{\Gamma(1-\alpha)} \int_0^t (t-\tau)^{-\alpha} y'(\tau) d\tau.$$

Our Goal: Convert the differential equation into an equivalent integral form. This is often the first step for numerical computation.

$$\text{FDE} \xrightarrow{\text{How?}} \text{Integral Equation}$$

The Key Tool: The Fractional Integral

To solve for $y(t)$, we need an operator that "undoes" the Caputo derivative. This is the **Riemann-Liouville fractional integral**, J^α .

Definition: Fractional Integral (J^α)

The fractional integral of a function $g(t)$ is defined as:

$$J^\alpha g(t) = \frac{1}{\Gamma(\alpha)} \int_0^t (t - \tau)^{\alpha-1} g(\tau) d\tau.$$

The Fundamental Property

The fractional integral is the **left-inverse** of the Caputo derivative. For $0 < \alpha < 1$, applying J^α to ${}^C D_t^\alpha$ yields:

$$J^\alpha ({}^C D_t^\alpha y(t)) = y(t) - y(0)$$

Derivation: From FDE to Volterra Equation (Steps 1-2)

Let's derive the integral form step-by-step.

① **Start with the FDE:**

$${}^C D_t^\alpha y(t) = f(t, y(t))$$

② **Apply the fractional integral J^α to both sides:**

$$J^\alpha ({}^C D_t^\alpha y(t)) = J^\alpha (f(t, y(t)))$$

Derivation: From FDE to Volterra Equation (Steps 3-4)

Continuing the derivation...

- ③ Use the fundamental property on the left-hand side:

$$y(t) - y(0) = J^\alpha (f(t, y(t)))$$

- ④ Substitute the definition of J^α and rearrange:

$$y(t) - y_0 = \frac{1}{\Gamma(\alpha)} \int_0^t (t - \tau)^{\alpha-1} f(\tau, y(\tau)) d\tau$$

Result: Equivalent Volterra Integral Equation

Isolating $y(t)$, we arrive at our final goal:

$$y(t) = y_0 + \frac{1}{\Gamma(\alpha)} \int_0^t (t - \tau)^{\alpha-1} f(\tau, y(\tau)) d\tau$$

The Core Formulas: PECE Method

The method solves $D^\alpha y(t) = f(t, y(t))$ by predicting a value y_{n+1}^P and then correcting it to find y_{n+1} .

Predictor (Adams-Bashforth)

$$y_{n+1}^P = \sum_{k=0}^{[\alpha]-1} \frac{t_{n+1}^k}{k!} y_0^{(k)} + \frac{h^\alpha}{\Gamma(\alpha)} \sum_{j=0}^n b_{n-j} f(t_j, y_j)$$

Corrector (Adams-Moulton)

$$y_{n+1} = \sum_{k=0}^{[\alpha]-1} \frac{t_{n+1}^k}{k!} y_0^{(k)} + \frac{h^\alpha}{\Gamma(\alpha)} \left(a_0 f(t_{n+1}, y_{n+1}^P) + \sum_{j=0}^n a_{n-j} f(t_j, y_j) \right)$$

Coefficients from the MATLAB Code

The weights, which depend only on the index j , are:

- Predictor weights: $b_j = j^\alpha - (j - 1)^\alpha$
- Corrector weights: $a_j = (j + 1)^\alpha - j^\alpha$

Linear FDE: Problem Setup

- **Equation:** $D^{0.5}y(t) = y(t)$ ($f(t, y) = y$)
- **Initial Condition:** $y(0) = 1$
- **Parameters:** $\alpha = 0.5$, $h = 0.01$

Goal: Find y_1, y_2, y_3 at times $t_1 = 0.01, t_2 = 0.02, t_3 = 0.03$.

Initial State ($n = 0$):

- $t_0 = 0, y_0 = 1$
- Initial function value: $f_0 = f(t_0, y_0) = y_0 = 1$

Linear FDE: Iteration 1

Computation at $t_1 = 0.01$

1. Predict (P)

$$y_1^P = y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [b_1 f_0] = 1 + 0.0564 \cdot [1 \cdot 1] = \mathbf{1.0564}$$

2. Evaluate (E)

$$f_1^P = f(t_1, y_1^P) = y_1^P = \mathbf{1.0564}$$

3. Correct (C)

$$y_1 = y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [a_0 f_1^P + a_1 f_0] = 1 + 0.0564 \cdot [1 \cdot 1.0564 + 0.4142 \cdot 1] = \mathbf{1.0830}$$

4. Evaluate (E) & Store History

$$f_1 = f(t_1, y_1) = y_1 = \mathbf{1.0830}$$

Linear FDE: Iteration 2

Computation at $t_2 = 0.02$

History known: $f_0 = 1$, $f_1 = 1.0830$

1. Predict (P)

$$y_2^P = y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [b_2 f_0 + b_1 f_1] = 1 + 0.0564 \cdot [0.4142 \cdot 1 + 1 \cdot 1.0830] = \mathbf{1.0845}$$

2. Evaluate (E)

$$f_2^P = f(t_2, y_2^P) = y_2^P = \mathbf{1.0845}$$

3. Correct (C)

$$y_2 = y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [a_0 f_2^P + a_1 f_1 + a_2 f_0] = 1 + 0.0564 \cdot [1 \cdot 1.0845 + \dots] = \mathbf{1.1042}$$

4. Evaluate (E) & Store History

$$f_2 = f(t_2, y_2) = y_2 = \mathbf{1.1042}$$

Linear FDE: Iteration 3

Computation at $t_3 = 0.03$

History known: $f_0 = 1$, $f_1 = 1.0830$, $f_2 = 1.1042$

1. Predict (P)

$$y_3^P = y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [b_3 f_0 + b_2 f_1 + b_1 f_2] = 1 + 0.0564 \cdot [0.3178 \cdot 1 + \dots] = \mathbf{1.1055}$$

2. Evaluate (E)

$$f_3^P = f(t_3, y_3^P) = y_3^P = \mathbf{1.1055}$$

3. Correct (C)

$$y_3 = y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [a_0 f_3^P + a_1 f_2 + a_2 f_1 + a_3 f_0] = 1 + 0.0564 \cdot [1 \cdot 1.1055 + \dots] = \mathbf{1.1215}$$

4. Evaluate (E) & Store History

$$f_3 = f(t_3, y_3) = y_3 = \mathbf{1.1215}$$

Non-Linear FDE: Problem Setup

- **Equation:** $D^{0.5}y(t) = y(t)(1 - y(t))$ ($f(t, y) = y(1 - y)$)
- **Initial Condition:** $y(0) = 0.5$
- **Parameters:** $\alpha = 0.5$, $h = 0.01$

Goal: Find y_1, y_2, y_3 at times $t_1 = 0.01, t_2 = 0.02, t_3 = 0.03$. **Initial State ($n = 0$):**

- $t_0 = 0$, $y_0 = 0.5$
- Initial function value: $f_0 = y_0(1 - y_0) = 0.25$

Non-Linear FDE: Iteration 1

Computation at $t_1 = 0.01$

1. Predict (P)

$$y_1^P = y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [b_1 f_0] = 0.5 + 0.0564 \cdot [1 \cdot 0.25] = \mathbf{0.5141}$$

2. Evaluate (E)

$$f_1^P = y_1^P(1 - y_1^P) = 0.5141(1 - 0.5141) = \mathbf{0.2498}$$

3. Correct (C)

$$\begin{aligned} y_1 &= y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [a_0 f_1^P + a_1 f_0] \\ &= 0.5 + 0.0564 \cdot [1 \cdot 0.2498 + 0.4142 \cdot 0.25] \\ &= \mathbf{0.5200} \end{aligned}$$

4. Evaluate (E) & Store History

$$f_1 = y_1(1 - y_1) = 0.5200(1 - 0.5200) = \mathbf{0.2496}$$

Non-Linear FDE: Iteration 2

Computation at $t_2 = 0.02$ **History known:** $f_0 = 0.25$, $f_1 = 0.2496$

1. Predict (P)

$$y_2^P = y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [b_2 f_0 + b_1 f_1] = 0.5 + 0.0564 \cdot [0.4142 \cdot 0.25 + \dots] = \mathbf{0.5199}$$

2. Evaluate (E)

$$f_2^P = y_2^P (1 - y_2^P) = 0.5199(1 - 0.5199) = \mathbf{0.2496}$$

3. Correct (C)

$$\begin{aligned} y_2 &= y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [a_0 f_2^P + a_1 f_1 + a_2 f_0] \\ &= 0.5 + 0.0564 \cdot [1 \cdot 0.2496 + \dots] \\ &= \mathbf{0.5244} \end{aligned}$$

4. Evaluate (E) & Store History

$$f_2 = y_2(1 - y_2) = 0.5244(1 - 0.5244) = \mathbf{0.2494}$$

Non-Linear FDE: Iteration 3

Computation at $t_3 = 0.03$ **History known:** $f_0 = 0.25$, $f_1 = 0.2496$, $f_2 = 0.2494$

1. Predict (P)

$$\begin{aligned}y_3^P &= y_0 + \frac{h^{0.5}}{\Gamma(0.5)} [b_3 f_0 + b_2 f_1 + b_1 f_2] \\&= 0.5 + 0.0564 \cdot [0.3178 \cdot 0.25 + \dots] \\&= \mathbf{0.5244}\end{aligned}$$

2. Evaluate (E)

$$f_3^P = y_3^P (1 - y_3^P) = 0.5244(1 - 0.5244) = \mathbf{0.2494}$$

3. Correct (C)

$$\begin{aligned}y_3 &= y_0 + \frac{h^{0.5}}{\Gamma(0.5)}[a_0 f_3^P + a_1 f_2 + a_2 f_1 + a_3 f_0] \\&= 0.5 + 0.0564 \cdot [1 \cdot 0.2494 + \dots] \\&= \mathbf{0.5283}\end{aligned}$$

4. Evaluate (E) & Store History

$$f_3 = y_3(1 - y_3) = 0.5283(1 - 0.5283) = \mathbf{0.2492}$$

Finite Difference Method for FDEs

What is the Finite Difference Method (FDM)?

FDM approximates derivatives in differential equations using discrete function values:

$$y'(t_n) \approx \frac{y(t_{n+1}) - y(t_n)}{h}$$

It transforms calculus problems into solvable algebraic systems.

Why standard ODE solvers (Euler, RK) fail for FDEs?

- **Integer-order derivatives are local:** Depend only on immediate vicinity.
- **Fractional-order derivatives are non-local:** Depend on the entire history of the function (i.e., they have "memory").

Standard ODE solvers cannot account for this "memory."

The Grünwald-Letnikov (GL) Approximation

To solve FDEs, we use methods that incorporate history. The GL definition provides a finite difference approximation for fractional derivatives:

$$D^\alpha y(t_n) \approx \frac{1}{h^\alpha} \sum_{j=0}^n w_j^{(\alpha)} y(t_{n-j})$$

GL Weights $w_j^{(\alpha)}$

These coefficients capture the "memory" effect:

- $w_0^{(\alpha)} = 1$
- $w_1^{(\alpha)} = -\alpha$
- $w_2^{(\alpha)} = \frac{\alpha(\alpha-1)}{2}$
- ... and recursively: $w_j^{(\alpha)} = \left(1 - \frac{\alpha+1}{j}\right) w_{j-1}^{(\alpha)}$, for $j \geq 1$

Example 1: Solving a Linear FDE

Problem Definition: Fractional Relaxation Equation

- **FDE:** $D^{0.8}y(t) = -y(t)$
- **Initial Condition:** $y(0) = 1$
- **Parameters:** Step size $h = 0.1$, fractional order $\alpha = 0.8$.

Discretization and Solution Formula

Replacing the fractional derivative $D^\alpha y(t_n)$ with the GL approximation and solving for y_n :

$$y_n \left(\frac{1}{h^\alpha} + 1 \right) = -\frac{1}{h^\alpha} \sum_{j=1}^n w_j^{(\alpha)} y_{n-j}$$

Leading to:

$$y_n = \frac{-1}{1 + h^\alpha} \sum_{j=1}^n w_j^{(\alpha)} y_{n-j}$$

Objective: Discretizing the FDE

The Fractional Differential Equation (FDE)

Our starting point is the following FDE:

$$D^{\alpha}y(t) = -y(t)$$

The Goal

We want to convert this continuous equation into its discrete form using the Grünwald-Letnikov (GL) approximation for the fractional derivative.

Step 1: Apply the Grünwald-Letnikov Formula

The GL approximation for the fractional derivative at a discrete time step t_n is:

$$D^\alpha y(t_n) \approx \frac{1}{h^\alpha} \sum_{j=0}^n w_j^{(\alpha)} y(t_{n-j})$$

We substitute this into our FDE, $D^\alpha y(t_n) = -y(t_n)$, and use the notation $y_k = y(t_k)$:

$$\frac{1}{h^\alpha} \sum_{j=0}^n w_j^{(\alpha)} y_{n-j} = -y_n$$

Step 2: Separate the Sum

From the previous step, we have:

$$\frac{1}{h^\alpha} \sum_{j=0}^n w_j^{(\alpha)} y_{n-j} = -y_n$$

To isolate the y_n term, we split the summation into its first term ($j = 0$) and the rest of the series ($j = 1$ to n).

$$\frac{1}{h^\alpha} \left(w_0^{(\alpha)} y_{n-0} + \sum_{j=1}^n w_j^{(\alpha)} y_{n-j} \right) = -y_n$$

Recall: GL Coefficients

The first GL coefficient is defined as $w_0^{(\alpha)} = 1$. Substituting this in gives:

$$\frac{1}{h^\alpha} \left(y_n + \sum_{j=1}^n w_j^{(\alpha)} y_{n-j} \right) = -y_n$$

Step 3: Rearrange and Factor out y_n

First, distribute the $\frac{1}{h^\alpha}$ term on the left-hand side:

$$\frac{1}{h^\alpha} y_n + \frac{1}{h^\alpha} \sum_{j=1}^n w_j^{(\alpha)} y_{n-j} = -y_n$$

Next, move all terms containing y_n to the left side and the summation to the right:

$$\frac{1}{h^\alpha} y_n + y_n = -\frac{1}{h^\alpha} \sum_{j=1}^n w_j^{(\alpha)} y_{n-j}$$

Final Form

Finally, factor out y_n on the left-hand side to arrive at the desired result:

$$y_n \left(\frac{1}{h^\alpha} + 1 \right) = -\frac{1}{h^\alpha} \sum_{j=1}^n w_j^{(\alpha)} y_{n-j}$$

Linear FDE: Weights and Iteration 1

Calculated GL Weights ($\alpha = 0.8$) and h^α

$$w_0 = 1$$

$$w_1 = -0.8$$

$$w_2 = -0.08$$

$$w_3 \approx 0.032$$

$$h^\alpha = 0.1^{0.8} \approx 0.1585$$

Iteration 1 ($n = 1$): Find $y_1 \approx y(0.1)$

$$y_1 = \frac{-1}{1 + h^\alpha} (w_1 y_0)$$

$$y_1 = \frac{-1}{1.1585} (-0.8 \times 1) \approx \mathbf{0.6905}$$

Linear FDE: Iterations 2 & 3

Iteration 2 ($n = 2$): Find $y_2 \approx y(0.2)$

$$y_2 = \frac{-1}{1.1585}(w_1 y_1 + w_2 y_0)$$

$$y_2 = \frac{-1}{1.1585}((-0.8 \times 0.6905) + (-0.08 \times 1))$$

$$y_2 = \frac{-1}{1.1585}(-0.5524 - 0.08) \approx \mathbf{0.5459}$$

Linear FDE: Iterations 2 & 3

Iteration 2 ($n = 2$): Find $y_2 \approx y(0.2)$

$$y_2 = \frac{-1}{1.1585}(w_1 y_1 + w_2 y_0)$$
$$y_2 = \frac{-1}{1.1585}((-0.8 \times 0.6905) + (-0.08 \times 1))$$
$$y_2 = \frac{-1}{1.1585}(-0.5524 - 0.08) \approx \mathbf{0.5459}$$

Iteration 3 ($n = 3$): Find $y_3 \approx y(0.3)$

$$y_3 = \frac{-1}{1.1585}(w_1 y_2 + w_2 y_1 + w_3 y_0)$$
$$y_3 = \frac{-1}{1.1585}((-0.8 \times 0.5459) + (-0.08 \times 0.6905) + (0.032 \times 1))$$
$$y_3 = \frac{-1}{1.1585}(-0.4367 - 0.0552 + 0.032) \approx \mathbf{0.4441}$$

Example 2: Solving a Non-Linear FDE

Problem Definition: Fractional Logistic Equation

- **FDE:** $D^{0.9}y(t) = y(t)(1 - y(t))$
- **Initial Condition:** $y(0) = 0.5$
- **Parameters:** Step size $h = 0.1$, fractional order $\alpha = 0.9$.

Discretization and Solution Formula (Explicit)

For simplicity in manual iteration, we use an explicit approach where the non-linear term is evaluated at the previous step y_{n-1} :

$$y_n = - \sum_{j=1}^n w_j^{(\alpha)} y_{n-j} + h^\alpha \cdot y_{n-1}(1 - y_{n-1})$$

The Non-Linear FDE and Our Goal

The Fractional Differential Equation (FDE)

We start with the non-linear FDE:

$$D^\alpha y(t) = y(t)(1 - y(t))$$

Objective

Our goal is to derive an explicit formula for y_n by applying the Grünwald-Letnikov (GL) approximation and handling the non-linear term. For this problem, we will use $\alpha = 0.9$.

Step 1: Apply the GL Formula

First, we discretize the equation at time $t = t_n$:

$$D^\alpha y(t_n) = y(t_n)(1 - y(t_n))$$

Next, we replace the fractional derivative with its GL approximation:

$$D^\alpha y(t_n) \approx \frac{1}{h^\alpha} \sum_{j=0}^n w_j^{(\alpha)} y_{n-j}$$

Substituting this into the FDE gives us:

$$\frac{1}{h^\alpha} \sum_{j=0}^n w_j^{(\alpha)} y_{n-j} = y_n(1 - y_n)$$

Step 2: Isolate the Current Term (y_n)

From the previous step, we have:

$$\frac{1}{h^\alpha} \sum_{j=0}^n w_j^{(\alpha)} y_{n-j} = y_n(1 - y_n)$$

We separate the $j = 0$ term from the sum, recalling that $w_0^{(\alpha)} = 1$:

$$\begin{aligned} \frac{1}{h^\alpha} \left(\underbrace{w_0^{(\alpha)} y_{n-0}}_{j=0} + \sum_{j=1}^n w_j^{(\alpha)} y_{n-j} \right) &= y_n(1 - y_n) \\ \Rightarrow \frac{1}{h^\alpha} \left(y_n + \sum_{j=1}^n w_j^{(\alpha)} y_{n-j} \right) &= y_n(1 - y_n) \end{aligned}$$

Distributing the leading term gives:

$$\frac{1}{h^\alpha} y_n + \frac{1}{h^\alpha} \sum_{j=1}^n w_j^{(\alpha)} y_{n-j} = y_n(1 - y_n)$$

Step 3: The Explicit Method Approximation

Our equation currently contains y_n on both sides, with one side being non-linear.

$$\frac{1}{h^\alpha} y_n = y_n(1 - y_n) - \frac{1}{h^\alpha} \sum_{j=1}^n w_j^{(\alpha)} y_{n-j}$$

The Explicit Approximation

To create an explicit method (which is easier to compute), we approximate the non-linear term on the right-hand side by evaluating it at the **previous time step**, t_{n-1} .

$$y_n(1 - y_n) \approx y_{n-1}(1 - y_{n-1})$$

Applying this approximation, our equation becomes linear in y_n :

$$\frac{1}{h^\alpha} y_n = y_{n-1}(1 - y_{n-1}) - \frac{1}{h^\alpha} \sum_{j=1}^n w_j^{(\alpha)} y_{n-j}$$

Step 4: Solve for y_n

We start with our approximated equation:

$$\frac{1}{h^\alpha} y_n = y_{n-1}(1 - y_{n-1}) - \frac{1}{h^\alpha} \sum_{j=1}^n w_j^{(\alpha)} y_{n-j}$$

To isolate y_n , we simply multiply the entire equation by h^α :

$$y_n = h^\alpha \left(y_{n-1}(1 - y_{n-1}) - \frac{1}{h^\alpha} \sum_{j=1}^n w_j^{(\alpha)} y_{n-j} \right)$$

$$y_n = h^\alpha \cdot y_{n-1}(1 - y_{n-1}) - \sum_{j=1}^n w_j^{(\alpha)} y_{n-j}$$

Non-Linear FDE: Weights and Iteration 1

Calculated GL Weights ($\alpha = 0.9$) and h^α

$$w_1 = -0.9$$

$$w_2 = -0.045$$

$$w_3 = 0.0285$$

$$h^\alpha = 0.1^{0.9} \approx 0.1259$$

Iteration 1 ($n = 1$): Find $y_1 \approx y(0.1)$

$$y_1 = -w_1 y_0 + h^\alpha \cdot y_0(1 - y_0)$$

$$y_1 = -(-0.9 \times 0.5) + 0.1259 \times (0.5 \times (1 - 0.5))$$

$$y_1 = 0.45 + 0.0315 = \mathbf{0.4815}$$

Non-Linear FDE: Iterations 2 & 3

Iteration 2 ($n = 2$): Find $y_2 \approx y(0.2)$

$$y_2 = -(w_1 y_1 + w_2 y_0) + h^\alpha \cdot y_1(1 - y_1)$$

$$y_2 = -((-0.9 \times 0.4815) + (-0.045 \times 0.5)) + 0.1259 \times (0.4815 \times (1 - 0.4815))$$

$$y_2 = -(-0.4334 - 0.0225) + 0.1259 \times (0.2496)$$

$$y_2 = 0.4559 + 0.0314 \approx \mathbf{0.4873}$$

Iteration 3 ($n = 3$): Find $y_3 \approx y(0.3)$

$$y_3 = -(w_1 y_2 + w_2 y_1 + w_3 y_0) + h^\alpha \cdot y_2(1 - y_2)$$

$$y_3 = -((-0.9 \times 0.4873) + (-0.045 \times 0.4815) + (0.0285 \times 0.5)) + 0.1259 \times (0.4873(1 - 0.4873))$$

$$y_3 = -(-0.4386 - 0.0217 + 0.0143) + 0.1259 \times (0.2498)$$

$$y_3 = 0.446 + 0.0314 \approx \mathbf{0.4774}$$

Python Code for Linear FDE Plot

The provided Python code implements the GL method for the linear FDE and plots it against the exact solution.

- Utilizes 'numpy' for efficient array operations.
- 'scipy.special.ml' is used for the Mittag-Leffler function (exact solution).
- Plotting is done with 'matplotlib.pyplot'.

Key sections of the code:

- `solve_linear_fde` function: Computes GL weights and iterates the solution.
- Exact solution calculation: Uses '`mittag_leffler(-tnumerical * *ALPHA, ALPHA)`'.
- Plotting with 'matplotlib' for clear visualization.

Linear FDE Plot

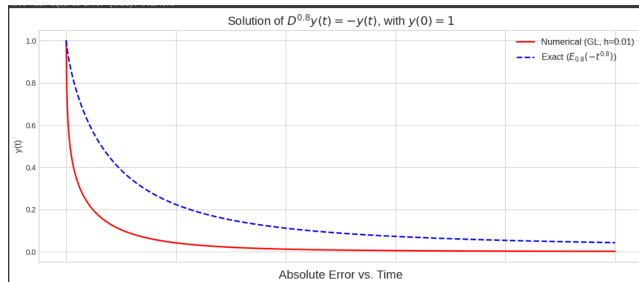


Figure: The exact solution v/s the approximated solution.

The Curious Case of the Error Curve

When solving the FDE $D^\alpha y(t) = -y(t)$, the numerical error exhibits a unique behavior.

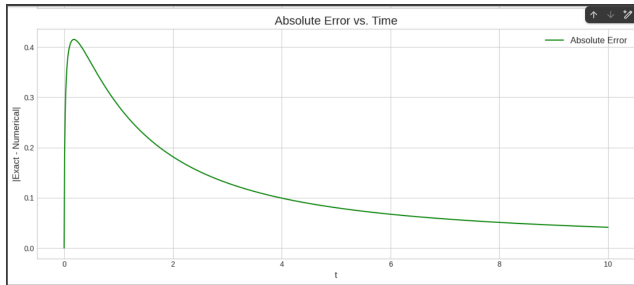


Figure: Absolute error between the exact and numerical solutions vs. time.

Key Observation

Instead of monotonically increasing, the error spikes rapidly at the beginning and then slowly decays over time. This talk explains why.

Phase 1: The "Short-Memory" Error Spike

The initial, sharp increase in error is caused by two factors:

- **Truncation of History:**

- ▶ The Grünwald-Letnikov derivative requires the function's **entire past history**:

$$D^\alpha y(t) \approx \frac{1}{h^\alpha} \sum_{j=0}^n w_j y(t_{n-j})$$

- ▶ At the start (small n), this sum provides a poor, "short-memory" approximation of a truly non-local operator.

- **Rapidly Changing Solution:**

- ▶ The exact solution, the Mittag-Leffler function $y(t) = E_\alpha(-t^\alpha)$, is very steep near $t = 0$.
- ▶ Any discrete method struggles to accurately capture this initial, rapid change.

In short...

The method has the least information precisely when the function is hardest to approximate.

Phase 2: "Long-Memory" Stabilization

The subsequent decay of the error is a hallmark of fractional systems.

- **The Memory Effect Improves Accuracy:**

- ▶ As t (and thus n) increases, the summation incorporates more and more past data points.
- ▶ The approximation's "memory" grows, allowing it to better capture the non-local nature of the FDE, which **reduces the error**.

- **The Solution Becomes Smoother:**

- ▶ For larger t , the solution $E_\alpha(-t^\alpha)$ becomes much smoother and changes more slowly.
- ▶ It is fundamentally easier for the numerical method to track a slow, smooth function.

Conclusion

The rise-and-fall error curve is a direct visualization of the **memory effect** in action. The error is highest when the method's memory is short, and it stabilizes as the memory grows and the system's behavior becomes less volatile.

Questions?